U.PORTO
FACULDADE DE CIÊNCIAS
UNIVERSIDADE DO PORTO

Academia de Física 2026

Experiência 1 – Capacidade calorífica [Guião de Resolução]

Neste documento encontrarás uma descrição do procedimento que deverás considerar durante a análise numérica dos dados obtidos experimentalmente para a determinação da dependência na temperatura da capacidade calorífica $\left(C_{sample}(T)\right)$ de uma amostra de hematite.

No Jupyter Notebook "Heat capacity – Notebook alunos" apenas necessitarás de modificar as células de código destacadas a vermelho.

(a) Extração dos dados

In [ ]:
```
# Temperatura da medição
T = 300 # K

# nome do ficheiro de dados
filename = directory + foldername + r'\Bulk'+str(T)+r'K.chi'

# load dos dados a partir do ficheiro
data = np.genfromtxt( filename , delimiter = ' ', skip_header = 1)

teta2_300 = data[:,0] # ângulo
I_300 = data[:,1] # Intensidade
```

(b) Identificação do máximo de intensidade

In [ ]:
```
# Identificação do ângulo e do máximo de intensidade
I_300_max = # máximo de intensidade
index_max = # índice associado ao máximo de intensidade
teta2_300_max = teta2_300[ index_max ] # ângulo associado ao máximo de intensidade

# Na representação de difractogramas é comum a sua normalização ao valor do máximo de intensidade
I_300_norm = I_300 / I_300_max
```

Figura 1: (a) Exemplo de célula a **não editar**. **(b)** Exemplo de célula a **editar.**

Descrição geral

Para determinar o comportamento de $C_{sample}(T)$, primeiramente é necessário efetuar a medição da adenda, obtendo assim a dependência da capacidade calorífica da *puck* com a *grease* $\left(C_{addenda}(T)\right)$. Em seguida, coloca-se a amostra sobre a *grease* e mede-se a capacidade calorífica total (C_{total}) do sistema *puck* + amostra para obter a dependência $C_{total}(T)$. A capacidade calorífica da amostra depois é obtida, simplesmente efetuando $C_{sample}(T) = C_{total}(t) - C_{addenda}(T)$.

No caso da determinação de $C_{addenda}(T)$, para cada valor de temperatura da gama em estudo, é necessário efetuar dois ajustes exponenciais à dependência temporal da temperatura da *puck* $\left(T_{puck}(t)\right)$, um em aquecimento $(P \neq 0)$ e outro em arrefecimento $(P = 0)$. Como podes imaginar, <u>se não for devidamente automatizado</u>, este processo será muito duradouro.

Posto isto, na **primeira parte** desta atividade tentaremos determinar a capacidade calorífica da adenda para uma temperatura do banho específica ($T_b = 320\ K$, e.g.), tendo em conta que depois,

na **segunda parte**, pretendemos automatizar o processo de análise para as restantes temperaturas. Na **terceira parte** iremos obter $C_{total}(T)$ (e por sua vez $C_{sample}(T)$) recorrendo ao método de *single curve slope analysis* (explicado adiante).

No **final**, é proposto um **desafio**, onde será aprofundada a análise de $C_{sample}(T)$, de modo a determinar a temperatura (T_t) e a energia (ΔE_t) associadas à transição de fase – parâmetros fundamentais para a integração dos materiais em contexto tecnológico.

Nota: Se tiveres dúvidas sobre como utilizar alguma das funções do *numpy* ou do *scipy* mencionadas adiante pesquisa pelo seu nome no Google, ou então chama um dos monitores.

Determinação da capacidade calorífica da adenda a $320\ K$

Parte 1 – Segmentação dos dados em aquecimento e arrefecimento

- Extração dos dados
  - Utilizando a função do *numpy* **np.genfromtxt()** extraímos os valores do ficheiro de dados "20250221_addenda_320.csv" para um *array* com o nome "*data*" com dimensões $N \times 3$ ($N \equiv$ nº de linhas, $3 \equiv$ nº de colunas).
    - Primeira coluna $\equiv$ valores de tempo (t)
    - Segunda coluna $\equiv$ valores da temperatura medida (T)
    - Terceira coluna $\equiv$ valores da potência debitada (P)
  - Com as linhas de código abaixo conseguimos então guardar os valores de t a $320\ K$ na variável "*t_addenda_320*" , os valores de T a $320\ K$ na variável "*T_addenda_320*" e os valores de P a $320\ K$ na variável "*P_addenda_320*".

```
# Atribuição da cada coluna a um array
t_addenda_320 = data[:,0] # tempo
T_addenda_320 = data[:,1] # Temperatura medida
P_addenda_320 = data[:,2] # Potência debitada
```

- **Segmentação dos dados em aquecimento e arrefecimento**
  - Utiliza a função do *numpy* **np.where()** para determinar o índice associado ao primeiro ponto medido onde a potência se anula ("*index_switch*"), ou seja, o índice onde ocorre a passagem do aquecimento para o arrefecimento
    - A instrução **np.where(condição a verificar)[0]** permite-nos obter os índices onde se verifica a condição que passamos como argumento. Neste caso, como queremos obter o primeiro índice onde a condição é verificada fazemos **np.where(condição)[0][0]**
    - Assim é possível determinar o instante onde ocorre a transição do aquecimento para o arrefecimento ("t_addenda_320_switch").
    - Este procedimento é feito pelas linhas de código:

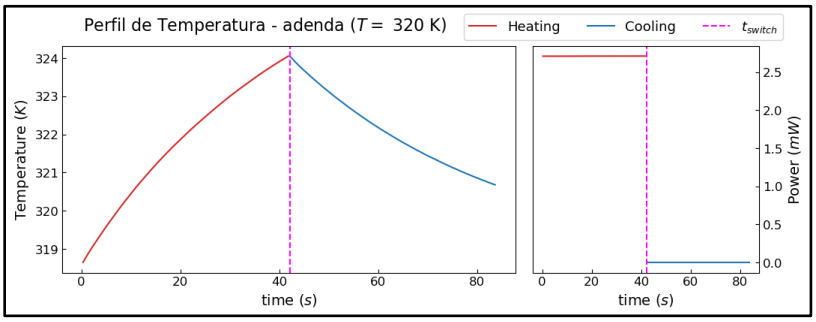
```
# Identificação do ponto onde a potência se torna nula
index_switch = np.where( P_addenda_320 == 0 )[0][0]
t_addenda_320_switch = t_addenda_320[ index_switch ]
```

- Ora, sabendo que o aquecimento é definido pelos índices que antecedem "*index_switch*" e que o arrefecimento é definido pelos índices que procedem "*index_switch*" podemos facilmente definir os *arrays* associados aos tempos, temperaturas e potências medidos durante o aquecimento e o arrefecimento.
  - Por exemplo, o array associado aos tempos de aquecimento ("*t_addenda_320_heating*") pode ser definido por:

```
# Segmentação dos dados - heating
t_addenda_320_heating = t_addenda_320[ : index_switch ]
```

  - Tendo isto em conta, preenche as restantes linhas da célula de código.

- Visualização geral dos dados
  - Resultado esperado:



Parte 2 – Determinação da capacidade calorífica

- **Funções de ajuste a considerar**
  - Define a funções **(heating_function** e **cooling_function)** que para o aquecimento e arrefecimento, respetivamente, introduzindo os valores de t, devolve os valores de ΔT que seguem uma curva exponencial com parâmetro $\Delta T(0)$, τ (e A no caso do aquecimento)
    - Apenas necessitas de associar "delta_T" à expressão apresentada no enunciado

- Função que efetua o ajuste considerando o método dos mínimos quadrados
  - Definição da função **(curve_fitting)** que efetua o ajuste aos dados experimentais

- **Ajuste do perfil de aquecimento**

- Começa por definir o array com os valores de $\Delta T = T - T_b$ durante o aquecimento (“*delta_T_addenda_320_heating*”)
  - Para o efeito deverás considerar que a temperatura do banho corresponde ao valor de temperatura medido no início do ensaio (“*T_addenda_320_heating[0]*”)

```
# Assumindo que no início do aquecimento a temperatura medida corresponde à temperatura do banho (T_b):
delta_T_addenda_320_heating = T_addenda_320_heating - T_addenda_320_heating[0]
```

- Determina os valores dos parâmetros de ajuste, utilizando a função **curve_fitting().** Deverás preencher os argumentos em falta.
  - *“parms_fit_heating”* – lista com os valores dos parâmetros de ajuste
  - *“u_parms_fit_heating”* – lista com os valores das incertezas dos parâmetros de ajuste
- Guarda os valores dos parâmetros de ajuste e suas incertezas em variáveis
  - Este procedimento é feito pelas seguintes linhas de código

```
# Determinação das constantes a partir dos parâmetros de ajuste
delta_T0_addenda_320_heating , tau_addenda_320_heating , A_addenda_320_heating = parms_fit_heating
u_delta_T0_addenda_320_heating , u_tau_addenda_320_heating , u_A_addenda_320_heating = u_parms_fit_heating
```

- Utilizando os valores determinados pelo ajuste, define o array corresponde ao valor de ΔT ajustado (“*delta_T_addenda_320_heating_fit*”)
- Avalia a qualidade do ajuste, calculando o coeficiente de Pearson (R^2). Essencialmente, quanto mais próximo de 1 for o valor de R^2 menor será o desvio dos valores de ajuste relativamente aos valores obtidos experimentalmente.
  - As células seguintes calculam este valor:

```
# Avaliação da qualidade do ajuste - Cálculo do coeficiente de Pearson (R^2)
r_addenda_320_heating , u_r_addenda_320_heating = \
            sps.pearsonr( delta_T_addenda_320_heating_fit , delta_T_addenda_320_heating )
R_addenda_320_heating = r_addenda_320_heating ** 2
# Quanto mais próximo de 1 for o valor de R^2, melhor a qualidade do ajuste
```

- Visualiza os valores dos parâmetros de ajuste.
  - Imprime os parâmetros de ajuste $\Delta T(0)$, τ guardados nas variáveis *delta_T0_addenda_320_heating* e *tau_addenda_320_heating,* respetivamente, assim como as suas incertezas.
  - Para isso, considera uma estrutura semelhante à seguinte linha de código:

```
# Print dos parâmetros de ajuste
print('\t Aquecimento')
print('Delta T_0 (K) = {:.2f} +- {:.2f}'.format(delta_T0_addenda_320_heating , u_delta_T0_addenda_320_heating))
```

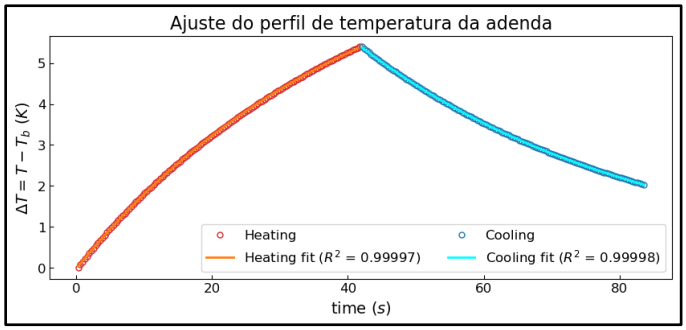
  - Resultados esperado:

```
        Aquecimento
Delta T_0 (K) = -0.03 +- 0.00
tau (s) = 42.17 +- 0.18
R^2 = 0.99997
-------------------------
```

- **Ajuste do perfil de arrefecimento**
    - Tal como no aquecimento, começa por definir o array com os valores de ΔT durante o arrefecimento ("*delta_T_addenda_320_cooling*")
        - Atenta que na expressão $\Delta T = T - T_b$, o valor de T_b deverá ser o mesmo que usaste para o aquecimento
    - De resto, tendo em conta as considerações tomadas para o ajuste do perfil de aquecimento, efetua as alterações necessárias para proceder ao ajuste durante o arrefecimento
    - Resultado esperado para os parâmetros do ajuste efetuado:

```
        Arrefecimento
Delta T_0 (K) = 5.38 +- 0.00
tau (s) = 42.48 +- 0.02
R^2 = 0.99998
------------------------
```

- Visualização do ajuste
    - Resultado esperado:



- **Determinação da capacidade calorífica**
    - Começa por determinar o valor da condutividade térmica do fio à temperatura da medição (temperatura do banho)
        - Para o efeito terás de usar algumas das variáveis definidas na célula inicial de "Extração da dependência de certos parâmetros com a temperatura do banho", nomeadamente "*T_addenda_T*" e "*K_w_addenda_T*" que te permitem obter a dependência $K_w(T_b)$
        - Utilizando a função **np.where()** determina o <u>primeiro índice</u> onde "*T_addenda_T*" corresponde à temperatura do banho (variável "*index_Tb*")
        - Utilizando esse índice, encontra o valor correspondente no array "*K_w_addenda_T*" e associa-o à variável "*K_w_addenda_Tb*"

- Considerando a fórmula apresentada para o cálculo da capacidade calorífica, recorre aos valores obtidos para τ em cada um dos ajustes e determina o seu valor em aquecimento ("*C_total_addenda_320_heating*"), arrefecimento ("*C_total_addenda_320_cooling*"), assim como a média dos dois ("*C_total_addenda_320*").
  - Resultado esperado:

```
Aquecimento - C_total (J/K) = 1.328e-02

Arrefecimento - C_total (J/K) = 1.338e-02

Valor médio - C_total (J/K) = 1.333e-02
```

Determinação da dependência de $C_{addenda}$ na temperatura

Nesta parte o objetivo passa por automatizar o processo de análise para as restantes temperaturas do banho, seguindo exatamente os mesmos passos considerados para a adenda a $320\ K$. Para o efeito, vamos tirar proveito da capacidade do Jupyter Notebook de armazenar na sua memória local dados (*array* ou um simples valor) associados a uma determinada variável (com um nome específico). Nesse sentido usaremos a seguinte estrutura:

- Quando quisermos guardar na memória um certo tipo de dados:

```python
# Nome de cada variável
t_addenda_name = 't_addenda_' + str(int(Tb))
T_addenda_name = 'T_addenda_' + str(int(Tb))
P_addenda_name = 'P_addenda_' + str(int(Tb))

# Alocação na memória dos arrays de dados
myVars = locals()
# myVars.__setitem__( nome da variável , array a associar à variável )
myVars.__setitem__( t_addenda_name , t_addenda )
myVars.__setitem__( T_addenda_name , T_addenda )
myVars.__setitem__( P_addenda_name , P_addenda )
```

- Quando quisermos utilizar o valor guardado anteriormente, atribuímo-lo a uma variável volátil (volátil porque muda de valor a cada iteração dos ciclos **for**):

```python
# Atribuição dos arrays previamente guardados a variáveis voláteis
# variavel volátil = globals()[ nome da variável na memória com os dados pretendidos ]
t_addenda_name = 't_addenda_' + str(int(Tb))
T_addenda_name = 'T_addenda_' + str(int(Tb))
P_addenda_name = 'P_addenda_' + str(int(Tb))

t_addenda = globals()[ t_addenda_name ]
T_addenda = globals()[ T_addenda_name ]
P_addenda = globals()[ P_addenda_name ]
```

Parte 3 – Obtenção da dependência da capacidade calorífica com a temperatura

<u>Segmentação dos dados em aquecimento e arrefecimento</u>

- **Extração dos dados**

- O procedimento abaixo é efetuado para cada iteração do ciclo **for**, ou seja, para cada temperatura "*Tb* " do array "*T_addenda_T*" (array com os valores de temperatura do banho considerados)
- Utilizando a função do *numpy* **np.genfromtxt()** extraímos os valores do ficheiro de dados "20250221_addenda_T_b.csv" para um *array* com o nome "*data*"
- Guardamos os valores de tempo, temperatura e potência nas variáveis voláteis "*t_addenda*", "*T_addenda*" e "*P_addenda*", respetivamente.
- Definimos o nome das variáveis que vamos guardar na memória associadas aos valores dos arrays voláteis
  - "*t_addenda_name*" – *string* com o nome da variável que guarda os valores medidos de tempo à temperatura T_b
  - "*T_addenda_name*" – *string* com o nome da variável que guarda os valores medidos de temperatura da amostra à temperatura T_b
  - "*P_addenda_name*" – *string* com o nome da variável que guarda os valores medidos de potência à temperatura T_b
- Guardamos essas variáveis na memória

- **Segmentação dos dados**
  - Para cada valor de temperatura T_b, atribuímos o correspondentes valores de tempo, temperatura e potência guardados na memória as variáveis voláteis "*t_addenda*", "*T_addenda*" e "*P_addenda*", respetivamente.
  - Utiliza a função do *numpy* **np.where()** para determinar o índice associado ao primeiro ponto medido onde a potência se anula ("*index_switch*"), ou seja, o índice onde ocorre a passagem do aquecimento para o arrefecimento. ***[a fazer]***
    - Recorda o procedimento que fizeste no caso da addenda a $320\ K$:

```
# Identificação do ponto onde a potência se torna nula
index_switch = np.where( P_addenda_320 == 0 )[0][0]
t_addenda_320_switch = t_addenda_320[ index_switch ]
```

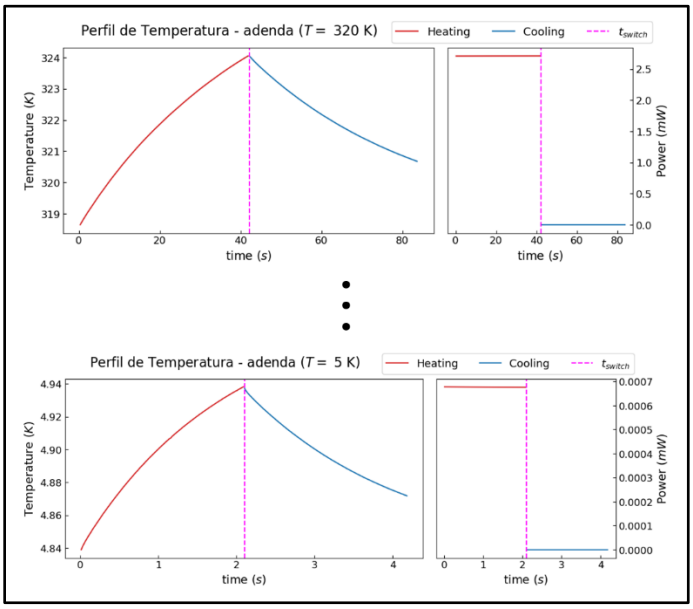
  - Ora, sabendo que o aquecimento é definido pelos índices que antecedem "*index_switch*" e que o arrefecimento é definido pelos índices que procedem "*index_switch*" podemos facilmente definir os *arrays* associados aos tempos, temperaturas e potências medidos durante o aquecimento e o arrefecimento. ***[a fazer]***
    - Recorda o que fizeste para determinar o array associado aos tempos de aquecimento ("*t_addenda_320_heating*"):

```
# Segmentação dos dados - heating
t_addenda_320_heating = t_addenda_320[ : index_switch ]
```

  - Definimos os nomes das variáveis que queremos guardar na memória, por exemplo, para o aquecimento definimos: (análogo para o arrefecimento)

- "*t_addenda_heating_name*" – *string* com o nome da variável que guarda os valores medidos de tempo à temperatura T_b, durante o aquecimento.
- "*T_addenda_heating_name*" – *string* com o nome da variável que guarda os valores medidos de temperatura da amostra à temperatura T_b, durante o aquecimento.
- "*P_addenda_heating_name*" – *string* com o nome da variável que guarda os valores medidos de potência à temperatura T_b, durante o aquecimento.

- Visualização geral dos dados
  - Resultado esperado:



Determinação da capacidade calorífica

- **Ajustes em aquecimento e arrefecimento**
  - Cria listas vazias onde guardarás os valores de parâmetros obtidos no ajuste às curvas em aquecimento e arrefecimento, em função da temperatura do banho
    - "*tau_addenda_heating_T*" – lista para guardar os valores de τ durante o aquecimento para cada T_b
    - "*tau_addenda_cooling_T*" – lista para guardar os valores de τ durante o arrefecimento para cada T_b
    - "*R_addenda_heating_T*" – *lista* com os valores de R^2 (coeficiente de Pearson) em função da temperatura, durante o arrefecimento
    - "*R_addenda_cooling_T*" – *lista* com os valores de R^2 (coeficiente de Pearson) em função da temperatura, durante o arrefecimento

- Para cada valor de temperatura T_b, começa por atribuir os valores guardados na memória (tempo e temperatura no aquecimento e arrefecimento) a variáveis voláteis (*arrays* "*t_addenda_heating*", "*t_addenda_cooling*", "*T_addenda_heating*" , "*T_addenda_cooling*"), respeitando nome que deste anteriormente a essas variáveis ("*t_addenda_heating_name*", "*t_addenda_cooling_name*", "*T_addenda_heating_name*" , "*T_addenda_cooling_name*")
- Define os *arrays* com os valores de $\Delta T = T - T_b$ durante o aquecimento e arrefecimento ("*delta_T_addenda_heating*" e "*delta_T_addenda_cooling*"))
  - Para o efeito deverás considerar que a temperatura do banho corresponde ao valor de temperatura medido no início do ensaio ("*T_addenda_heating[0]*")
- Procede ao ajuste em aquecimento
  - Determina os valores dos parâmetros de ajuste, utilizando a função **curve_fitting().** Deverás preencher os argumentos em falta.
    - *"parms_fit_heating"* – lista com os valores dos parâmetros de ajuste
    - *"u_parms_fit_heating"* – lista com os valores das incertezas dos parâmetros de ajuste
  - Guarda os valores dos parâmetros de ajuste e suas incertezas em variáveis
    - Recorda como fizeste isto para o caso em que $T_b = 320\ K$

```
# Determinação das constantes a partir dos parâmetros de ajuste
delta_T0_addenda_320_heating , tau_addenda_320_heating , A_addenda_320_heating = parms_fit_heating
u_delta_T0_addenda_320_heating , u_tau_addenda_320_heating , u_A_addenda_320_heating = u_parms_fit_heating
```

  - Utilizando os valores determinados pelo ajuste, define o array corresponde ao valor de ΔT ajustado em aquecimento ("*delta_T_addenda_heating_fit*")
  - Avalia a qualidade do ajuste, calculando o coeficiente de Pearson (R^2). Essencialmente, quanto mais próximo de 1 for o valor de R^2 menor será o desvio dos valores de ajuste relativamente aos valores obtidos experimentalmente.
    - As células seguintes calculam este valor:

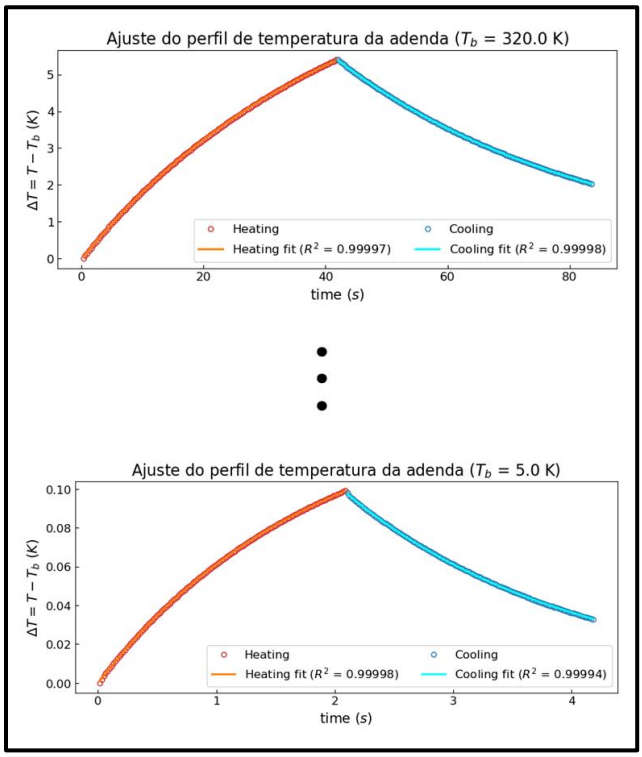
```
    # Avaliação da qualidade do ajuste - Cálculo do coeficiente de Pearson
    r_addenda_heating , u_r_addenda_heating = \
                sps.pearsonr( delta_T_addenda_heating_fit , delta_T_addenda_heating )
    R_addenda_heating = r_addenda_heating ** 2
```

  - Visualiza os valores dos parâmetros de ajuste.
    - Imprime os parâmetros de ajuste $\Delta T(0)$, τ guardados nas variáveis *delta_T0_addenda_heating e tau_addenda_heating,* respetivamente, assim como as suas incertezas.
    - Para isso, considera uma estrutura semelhante à seguinte linha de código:

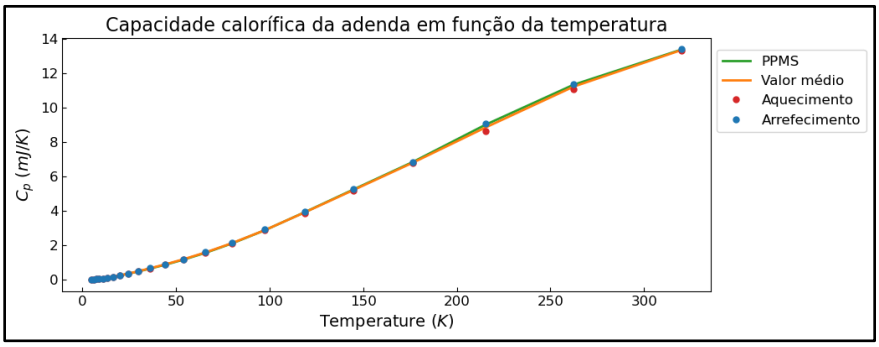
```
# Print dos parâmetros de ajuste
print('\t Aquecimento')
print('Delta T_0 (K) = {:.2f} +- {:.2f}'.format(delta_T0_addenda_320_heating , u_delta_T0_addenda_320_heating))
```

- Tendo por base o caso do aquecimento, procede ao ajuste em arrefecimento ***[a fazer]***
- Adicionamos os parâmetros obtidos nos ajustes (τ e R^2) às listas criadas anteriormente utilizar o método **lista.append()**
  - Como argumento deverás passar a variável associada ao valor de cada parâmetro
- Definimos os nomes das variáveis que queremos guardar na memória, por exemplo, para o aquecimento definimos: (análogo para o arrefecimento)
  - "*delta_T_addenda_heating_name*" – *string* com o nome da variável que guarda os valores medidos de ΔT em função do tempo à temperatura T_b, durante o aquecimento.
  - "*delta_T_addenda_heating_fit_name*" – *string* com o nome da variável que guarda os valores de ajuste de ΔT em função d006F tempo à temperatura T_b, durante o aquecimento.
- Guardamos as variáveis na memória.

- Visualização gráfica dos ajustes
  - Desejamos ver as dependências temporais dos valores medidos e ajustados de ΔT em aquecimento e arrefecimento.
    - Para isso deverás preencher adequadamente as abcissas e as ordenadas na secção de plotting.
  - Resultado esperado:



- **Determinação da capacidade calorífica**
    - Começa por transformar as listas com os valores de $\tau(T_b)$ em aquecimento e arrefecimento, em *arrays*, utilizando a função do numpy **np.array()**
    - Recorrendo à fórmula apresentada anteriormente, determina os valores da capacidade calorífica em aquecimento e arrefecimento
        - Para teres em conta a dependência da condutividade térmica do fio na temperatura, deverás considerar o array "*K_w_addenda_T*"
- Visualização da dependência da capacidade calorífica na temperatura
    - Resultado esperado:



Determinação da dependência de C_{total} na temperatura

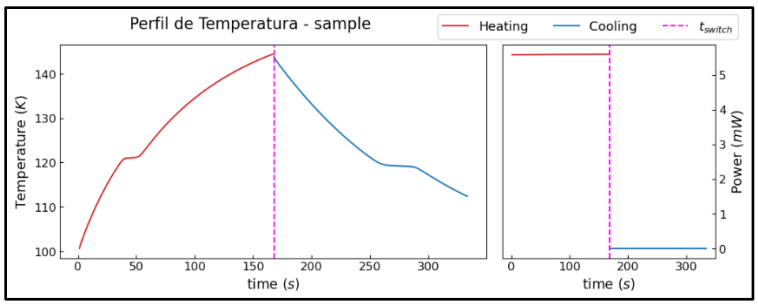
Parte 4 - Obtenção da dependência da capacidade calorífica da amostra com a temperatura.

Parte 4a – Segmentação dos dados em aquecimento e arrefecimento

- **Extração dos dados**
    - Utilizando os métodos até então usados para a extração dos dados a partir de ficheiros, atribui cada coluna do ficheiro à respetiva variável.
        - "*t_sample*" – array com os tempos da medição (1ª coluna)
        - "*T_sample*" – array com a variação temporal da temperatura (2ª coluna)
        - "*P_sample*" – array com a variação temporal da potência (3ª coluna)

- **Segmentação dos dados**
    - Mantendo a mesma lógica durante a análise da adenda, começa por determinar o índice a partir do qual a potência se anula ("*index_switch*") e, recorrendo a esse índice, segmenta os arrays "*t_sample*", "*T_sample*" e "*P_sample*" em aquecimento e arrefecimento.

- Visualização geral dos dados
    - Resultado esperado:



Parte 4b – Obtenção da dependência do declive na temperatura

- **Cálculo da derivada**
  - O procedimento para aquecimento e arrefecimento é análogo, por isso apenas se explicará o caso do aquecimento e depois deverás tentar replicar para o arrefecimento.
  - Começa por criar o array "*dT_dt_sample_heating*" onde armazenarás os valores da derivada da temperatura durante o aquecimento. Para isso, voltamos a utilizar a função **np.zeros()**.
    - Nota que, sendo N a dimensão do novo *array* dos tempos a dimensão de cada array com os valores da derivada é $N - 1$, uma vez que mantendo a fórmula do enunciado, não é possível calcular a derivada no último ponto.
  - Para calcular o valor da derivada em cada ponto, poderás definir um **ciclo for** onde percorrerás os pontos adquiridos durante o aquecimento.
    - Utiliza a fórmula do enunciado para calcular o i-ésimo valor da derivada.
    - Podes proceder da seguinte forma:

```
# Calcula a derivada em cada ponto para o aquecimento
for i in range( len(dT_dt_sample_heating) ):

    dT_dt_sample_heating[i] = \
    ( T_sample_heating[i+1] - T_sample_heating[i] ) / \
    ( t_sample_heating[i+1] - t_sample_heating[i] )
```

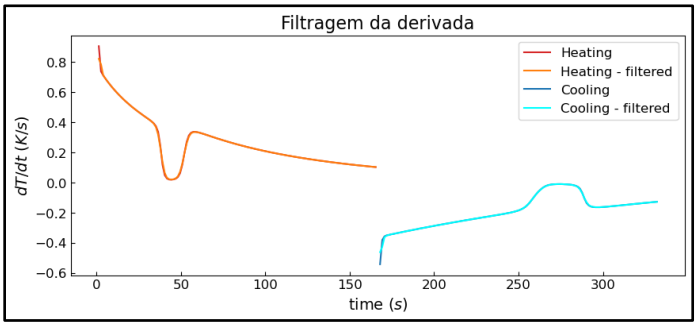
  - Repete o processo para o arrefecimento

- Função de filtragem – médias móveis
  - Define a função **(medias_moveis)** que ao introduzir um array (*dados*) efetua uma filtragem de médias móveis com um bloco de dimensão $2r + 1$

- **Filtragem da derivada**
  - Define a percentagem de primeiros vizinhos a considerar (variável "*p_neighbours*")
  - Para o aquecimento:
    - Define a dimensão do bloco de filtragem a partir da variável "*r_filter_sample_heating*"

        - Utilizando a função **medias_moveis()** aplica um filtro à derivada para determinares a dependência $S(T)$
    - Para o arrefecimento, repete o mesmo procedimento do aquecimento

- Visualização gráfica da derivada e de $S(T)$
    - Resultado esperado:

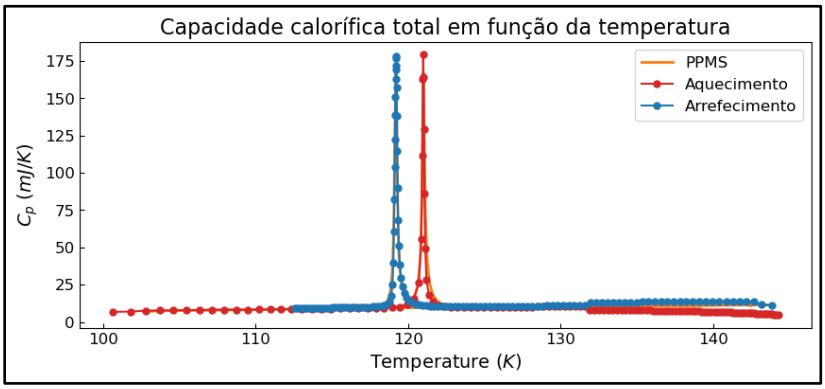


Parte 4c – Determinação do valor da capacidade calorífica total

- Extração dos resultados obtidos pelo MultiVu
    - Para efeitos comparativos, extraímos os valores obtidos ao analisar as curvas com o software incorporado no PPMS (MultiVu)

- **Determinação da dependência de K_w na temperatura**
    - Como os valores de temperatura estudados na medição da capacidade calorífica da amostra são diferentes dos valores medidos durante a adenda, é necessário efetuar uma aproximação que nos permita obter a dependência $K_w(T)$
        - Essencialmente, para cada valor de temperatura T_b medido para a amostra, vamos procurar na adenda o valor de K_w associado à temperatura mais próxima de T_b
    - Para o aquecimento:
        - Criamos o array "*K_w_sample_T_heating*" usando a função **np.zeros()**
        - Utilizando o **ciclo for** percorremos os valores de temperatura do aquecimento.
            - Para cada temperatura, determinamos o índice ("*index*") que respeita a condição pretendida
            - Atribuímos o valor de "*K_w_addenda_T*" associado a esse índice ao i-ésimo elemento de "*K_w_sample_T_heating*"
    - Para o arrefecimento segue a mesma metodologia do aquecimento

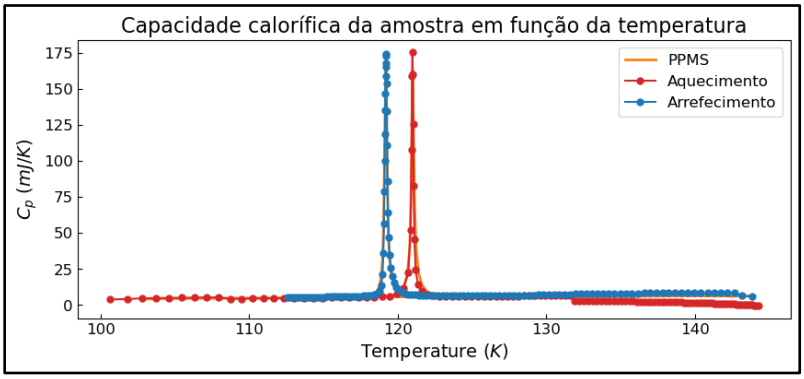
- **Determinação da capacidade calorífica total**
    - Considerando a expressão apresentada na introdução da Parte 4, calcula a capacidade calorífica total no aquecimento e no arrefecimento
        - Deverás assumir que a temperatura do banho corresponde à temperatura inicial do aquecimento
- Visualização da dependência da capacidade calorífica total na temperatura
    - Resultado esperado:



Parte 4d – Determinação do valor da capacidade calorífica da amostra

- **Determinação da dependência da C_{adenda} na temperatura**
    - Pelos mesmos motivos mencionados na parte 4c, temos de efetuar uma aproximação exatamente análoga para obter
    - Segue o mesmo procedimento nesta parte, quer para o aquecimento, quer para o arrefecimento

- **Determinação e visualização da capacidade calorífica da amostra (C_{sample})**
    - Calcula a capacidade calorífica da amostra em aquecimento e arrefecimento $(C_{sample} = C_{total} - C_{addenda})$
    - Resultado esperado:



Desafio final – Determinação da temperatura e da energia da transição de fase

Neste desafio, o objetivo é testar a tua independência e pensamento crítico, analisando de forma mais detalhada as transições de fase detetadas na medição da capacidade calorífica. Entre as informações possíveis de extrair, propõe-se a determinação de três parametros cruciais para a utilização de materiais com transições de fase em contexto tecnológico: a temperatura de transição (T_t), a largura da janela da transição (δT) e a energia da transição (ΔE_t).

Uma análise congruente deverá determinar parâmetros em questão de forma sequencial. Durante o processo deverás tentar responder às seguintes questões:

- Qual a forma mais expedita de determinar T_t ? E qual a mais adequada?
- Qual a forma mais expedita de determinar δT ? E qual a mais adequada?
- Como é que T_t e δT podem afetar a determinação do valor de E_t ?

Para referência, os valores de ΔE_t deverão rondar os $30 - 40\ mJ$.

Reitera-se que este desafio visa testar as tuas aprendizagens no final das sessões computacionais. No entanto, sempre que tiveres dúvidas podes e deves chamar os teus monitores. Boa sorte!